

INTERNSHIP PROJECT REPORT

THREAT INTELLIGENCE AGGREGATOR

(NON-AI)

Prepared by

Nitin Sukthe

Cybersecurity Internship Project

August 2026

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Scope
6. Technologies and Tools
7. Threat Intelligence Sources
8. System Architecture
9. Project Workflow
10. Module Implementation
11. IOC Validation and Normalization
12. Correlation Engine
13. Risk Scoring Engine
14. Blocklist Generation
15. Web Dashboard
16. Testing and Validation
17. Results
18. Security Considerations
19. Limitations
20. Future Scope
21. Conclusion
22. References

1. Abstract

The Threat Intelligence Aggregator is a non-AI cybersecurity application designed to collect and process Indicators of Compromise (IOCs) from multiple public threat intelligence feeds. The system automates feed collection, IOC parsing, validation, normalization, duplicate removal, cross-source correlation, explainable risk scoring, blocklist generation, reporting, and dashboard visualization. The implementation uses Python and Flask and is intentionally rule-based so that each processing and prioritization decision can be explained to a security analyst.

During development, the system successfully processed two live public feeds: Feodo Tracker and Phishunt. A latest development run processed 2,342 raw IOC records and produced 1,879 unique normalized indicators. The run generated 1,557 HIGH-risk and 322 MEDIUM-risk indicators, along with IP, domain, and URL blocklists. A separate reproducible correlation test verifies that indicators appearing in two or more sources are correctly identified and prioritized.

2. Introduction

Threat intelligence is valuable to Security Operations Center (SOC) and Blue Team workflows because it provides indicators that can be used for threat hunting, detection, blocking, and investigation. In practice, intelligence often arrives from multiple sources with different formats and different metadata conventions. A defensive engineering tool must therefore standardize the input before useful correlation and prioritization can take place.

This project implements that workflow without artificial intelligence or machine learning. The design focuses on transparent processing rules, predictable outputs, and practical artifacts that can be reviewed by a security analyst.

3. Problem Statement

Threat feeds may provide IP addresses, domains, URLs, hashes, and other indicators in TXT, CSV, and JSON formats. Without an aggregation and normalization layer, analysts may have to manually merge files, remove duplicates, determine indicator types, compare sources, and prepare blocklists. This creates repetitive work and makes consistent prioritization more difficult.

The project addresses this problem by creating a single defensive pipeline that turns heterogeneous feed data into a unified IOC dataset and actionable exports.

4. Objectives

- Collect threat intelligence from multiple feeds and local files.
- Parse IP addresses, domains, URLs, hashes, and email indicators.
- Validate and normalize heterogeneous IOC values.
- Remove duplicate indicators while preserving source information.
- Correlate indicators that appear in multiple sources.
- Assign explainable LOW, MEDIUM, or HIGH risk levels.

- Generate IP, domain, URL, and hash blocklists.
- Export machine-readable JSON and CSV results.
- Provide a web dashboard for investigation and download operations.
- Generate a reusable threat-intelligence summary report.

5. Scope

- In scope: IOC ingestion, parsing, validation, normalization, correlation, scoring, reporting, blocklist generation, and dashboard presentation.
- In scope: Feodo Tracker and Phishunt as live feed integrations.
- In scope: optional URLhaus integration using an environment-based Auth-Key.
- Out of scope: exploitation of real systems, execution of malware, and automated enforcement on production firewalls or endpoints.

6. Technologies and Tools

Technology / Tool	Purpose
Python 3	Core implementation language and pipeline orchestration.
Flask	Web dashboard and download endpoints.
Requests	HTTP collection of external feed data.
python-dotenv	Environment-variable and optional secret handling.
Regular Expressions (re)	IOC pattern detection and validation.
ipaddress	IPv4/IPv6 validation.
CSV / JSON	Feed parsing and machine-readable exports.
Git / GitHub	Version control and project submission.
HTML / CSS	Dashboard and generated report presentation.

7. Threat Intelligence Sources

7.1 Feodo Tracker

Feodo Tracker publishes botnet command-and-control (C2) IP blocklists and IOC datasets. The official documentation states that its botnet C2 IP data can be used for defensive blocking and that its IOC datasets include additional information about tracked C2s. The project consumes a CSV IOC feed and preserves first-seen, last-online, port, status, malware, and source metadata. [1]

7.2 Phishunt

Phishunt provides open feeds of active suspicious phishing and scam sites in TXT, JSON, and CSV formats. Its current feed documentation describes fields including URL, domain, company, date, and first_seen, with the feeds refreshed hourly. The project uses the CSV feed to extract URL, domain, and resolved-IP indicators. [2]

7.3 URLhaus Integration

The project also includes an optional URLhaus collector. Current URLhaus API documentation requires an Auth-Key for dataset downloads; therefore, the key is loaded from an environment variable and is never embedded in source code. [3]

8. System Architecture

The high-level architecture of the implemented system is:



The architecture is deliberately modular. Feed-specific collectors and parsers can be added without changing the core validation, normalization, correlation, or reporting logic.

9. Project Workflow

Workflow Step	Implementation
Step 1 - Load Feeds	Download or load threat intelligence data from supported feeds.

Step 2 - Parse Indicators	Extract supported IOC values and relevant metadata.
Step 3 - Validate	Reject malformed or unsupported IOC values.
Step 4 - Normalize	Standardize values and merge duplicate indicators.
Step 5 - Correlate	Count independent reporting sources for each normalized IOC.
Step 6 - Risk Score	Apply transparent rules using source count, IOC type, confidence, and category.
Step 7 - Generate Blocklists	Export medium- and high-risk IOCs by type.
Step 8 - Report	Generate a human-readable report and expose results in the dashboard.

10. Module Implementation

Feed Collectors

The collector layer handles external feed retrieval and saves raw data under data/raw. HTTP downloads use timeouts and a project user-agent and report failures without exposing secrets.

Parsers

Generic parsers handle TXT, CSV, and JSON input, while feed-specific parsers preserve provider-specific metadata for Feodo Tracker and Phishunt.

Core Validation

The validator detects IPs, domains, URLs, MD5, SHA1, SHA256, and email indicators. IPv4/IPv6 validation uses Python ipaddress and hash detection uses hexadecimal length/pattern rules.

Normalization

The normalizer standardizes case-sensitive and case-insensitive values as appropriate and merges identical (type, indicator) pairs while retaining the list of sources.

Correlation

The correlator adds source_count and correlation_level. One source is LOW, two sources are MEDIUM, and three or more sources are HIGH for correlation purposes.

Risk Scoring

The scoring engine adds transparent points for source count, IOC type, confidence, and high-risk categories, then maps the final score to LOW, MEDIUM, or HIGH.

Blocklist Generator

The generator creates separate TXT lists for IPs, domains, URLs, and hashes and also creates CSV and JSON outputs for downstream use.

Reporting

The reporting module creates an HTML summary with feed statistics, risk distribution, IOC type distribution, correlation results, high-risk indicators, methodology, and conclusion.

Dashboard

The Flask application displays statistics, provides search and filters, supports pagination, executes the pipeline, and exposes controlled download endpoints.

11. IOC Validation and Normalization

The project first determines the type of an indicator and rejects unsupported values. Example transformations include:

Input	Detected Type	Normalized Form
8.8.8.8	IP	8.8.8.8
Example.COM	DOMAIN	example.com
HTTPS://Example.com/	URL	https://example.com
d41d8cd98f00b204e9800998ecf8427e	MD5	lowercase MD5
test@example.com	EMAIL	test@example.com

12. Correlation Engine

The correlation engine is implemented as a source-counting mechanism over normalized IOC records. If an indicator is reported by multiple feeds, the source list is preserved and `source_count` is increased. This provides a simple, explainable prioritization signal.

The live-feed run on 16 August 2026 produced 0 cross-feed correlations because the Feodo Tracker and Phishunt datasets did not contain the same normalized indicators during that run. This is a valid real-world outcome and is not treated as an error.

A separate reproducible test fixture verifies the correlation logic using reserved/example indicators. The test confirms a three-source IOC receives HIGH correlation and a two-source IOC receives MEDIUM correlation.

13. Risk Scoring Engine

The rule-based scoring model is intentionally explainable. The implemented scoring factors include:

Factor	Rule / Contribution
Source count	1 source: +10; 2 sources: +30; 3+ sources: +50
IOC type	IP +10; URL +15; file hash +15; domain +10; email +5
Confidence	80+ confidence: +20; 50-79: +10
High-risk category	Categories such as malware, botnet, C2, phishing,

	ransomware, malicious: +20
Severity mapping	60+ HIGH; 30-59 MEDIUM; below 30 LOW

Because the scoring model is rule-based, an analyst can inspect risk_reasons for an IOC and understand why it was prioritized.

14. Blocklist Generation

The system exports medium- and high-risk indicators into separate blocklists suitable for defensive review and controlled deployment.

Output	Purpose
ip_blocklist.txt	Network / firewall IP deny list.
domain_blocklist.txt	Web filtering / DNS policy list.
url_blocklist.txt	Web proxy / URL filtering list.
hash_blocklist.txt	Endpoint/EDR/AV file-hash list.
blocklist.csv	Tabular machine-readable export with metadata.
blocklist.json	Structured machine-readable export.

15. Web Dashboard

The Flask dashboard provides a browser-based view of the latest pipeline output. It includes summary cards, feed and IOC statistics, blocklist downloads, searchable indicators, IOC-type and risk filters, pagination, and a button to execute the pipeline.

16. Testing and Validation

Test	Expected Behavior	Observed Result
IOC type detection	Identify IP/domain/URL/hash/email types.	Passed on representative samples.
TXT parser	Parse valid lines and ignore invalid values.	Passed.
CSV parser	Extract IOC-like values from rows.	Passed.
JSON parser	Recursively extract IOC-like strings.	Passed.
Feodo parser	Preserve IP and malware metadata.	Passed.
Phishunt parser	Extract URL/domain/IP records.	Passed on live feed run.
Correlation test	3 sources -> HIGH; 2 sources -> MEDIUM.	Passed.
Pipeline	Collection -> reporting without manual intermediate steps.	Passed.
Blocklists	Generate TXT/CSV/JSON outputs.	Passed.

Dashboard filters	Search and filter result set with pagination.	Passed.

17. Results

The latest recorded development run shown in the project evidence was generated on 16 August 2026 at 12:28:20 and reported the following values:

Metric	Observed Value
Feeds processed	2
Raw IOCs	2,342
Unique IOCs	1,879
Cross-feed correlated IOCs	0 in this live run
HIGH risk	1,557
MEDIUM risk	322
LOW risk	0
IP blocklist entries	322
Domain blocklist entries	778
URL blocklist entries	779
Hash blocklist entries	0

These counts are a snapshot of one development run. Because external threat feeds are dynamic, future runs may produce different totals.

18. Security Considerations

- API credentials are stored in environment variables and excluded from Git version control.
- Live feed files are excluded from the repository because they are dynamic and can become stale.
- Threat indicators are treated as untrusted data and are not executed by the application.
- Generated blocklists are intended for defensive review and should be validated before production enforcement.
- Download endpoints allow only an explicit list of generated files.

19. Limitations

- Feed availability and schema are controlled by external providers.
- Cross-feed correlation depends on actual indicator overlap; zero overlap is possible in a valid run.
- Current persistence is file-based and does not provide historical database tracking.

- The scoring model is a simple rule-based model designed for transparency rather than production-grade risk intelligence.
- The dashboard currently operates as a local Flask application; public deployment is a separate step.

20. Future Scope

- Add STIX/TAXII ingestion and richer threat-intelligence object mapping.
- Add scheduled feed collection and feed health monitoring.
- Add SQLite or PostgreSQL persistence and historical IOC tracking.
- Integrate with SIEM and ticketing platforms.
- Add authentication and role-based access control.
- Add configurable risk-scoring rules and confidence calibration.
- Add REST endpoints for downstream security controls.
- Add automated expiration and lifecycle management for old indicators.

21. Conclusion

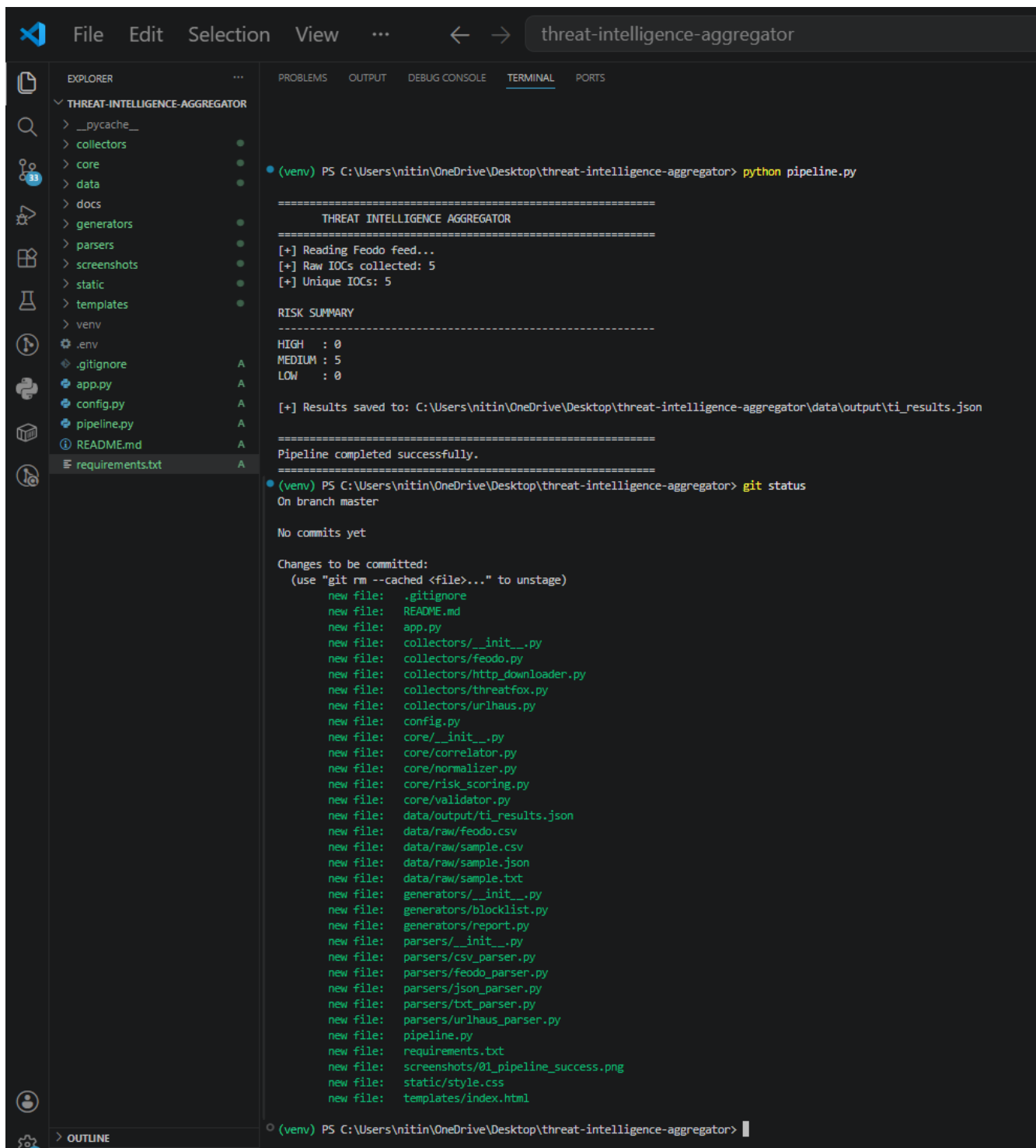
The Threat Intelligence Aggregator successfully demonstrates a practical, non-AI threat-intelligence workflow. The project collects data from multiple sources, validates and normalizes heterogeneous indicators, correlates repeated values, calculates transparent risk scores, generates blocklists, creates an HTML intelligence report, and exposes the results through a Flask dashboard.

The implementation emphasizes explainability and defensive usability. The project also includes reproducible tests for the correlation engine, Git-based version control, documentation, and evidence screenshots, making it suitable as an internship project demonstrating SOC and Blue Team automation skills.

22. References

- [1] Feodo Tracker. Blocklist and IOC documentation. <https://feodotracker.abuse.ch/blocklist/>
- [2] Phishunt. Phishing domains feed and API documentation. <https://phishunt.io/feed/> and <https://phishunt.io/api/>
- [3] URLhaus. Community API documentation and Auth-Key requirement. <https://urlhaus.abuse.ch/api/>

Appendix A - Project Evidence 1



```
File Edit Selection View ... threat-intelligence-aggregator

EXPLORER
THREAT-INTELLIGENCE-AGGREGATOR
  > __pycache__
  > collectors
  > core
  > data
  > docs
  > generators
  > parsers
  > screenshots
  > static
  > templates
  > venv
  .env
  .gitignore
  app.py
  config.py
  pipeline.py
  README.md
  requirements.txt

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(venv) PS C:\Users\nitin\OneDrive\Desktop\threat-intelligence-aggregator> python pipeline.py

=====
THREAT INTELLIGENCE AGGREGATOR
=====
[+] Reading Feodo feed...
[+] Raw IOCs collected: 5
[+] Unique IOCs: 5

RISK SUMMARY
-----
HIGH : 0
MEDIUM : 5
LOW : 0

[+] Results saved to: C:\Users\nitin\OneDrive\Desktop\threat-intelligence-aggregator\data\output\ti_results.json

=====
Pipeline completed successfully.
=====

(venv) PS C:\Users\nitin\OneDrive\Desktop\threat-intelligence-aggregator> git status
On branch master

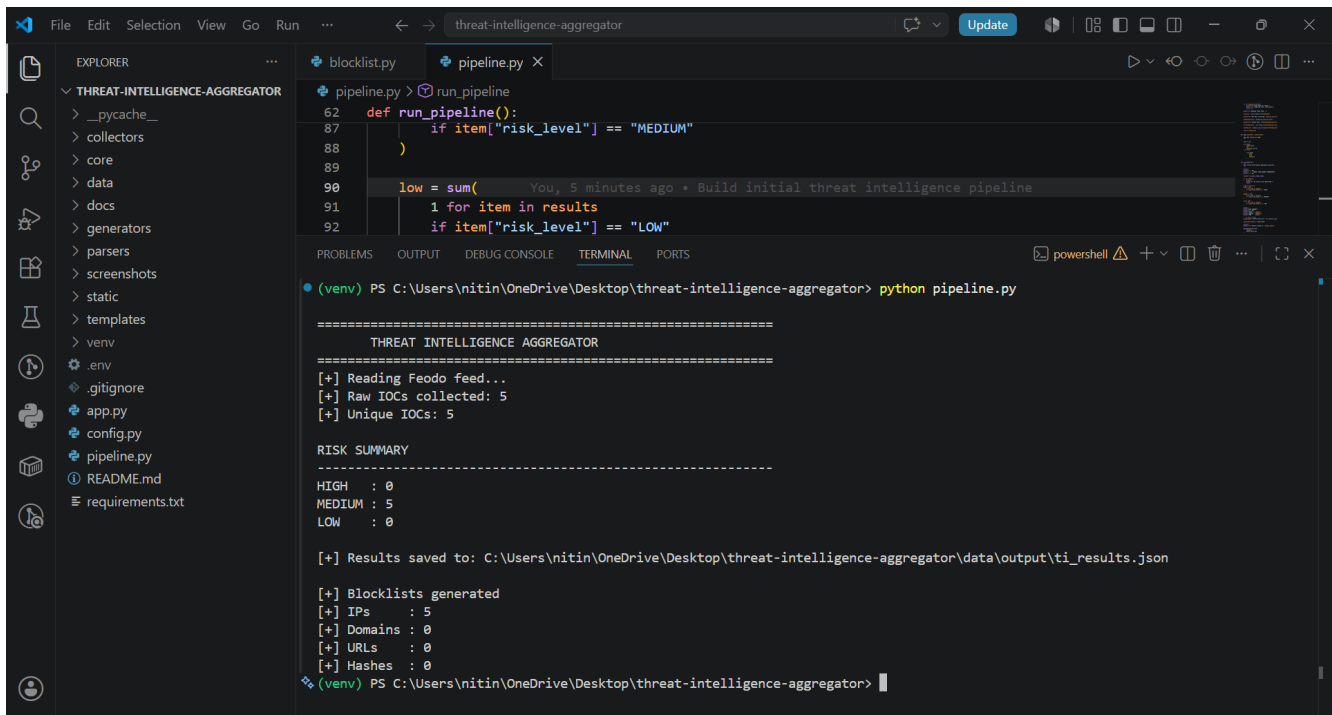
No commits yet

Changes to be committed:
(use "git rm --cached <file>..." to unstage)
    new file:   .gitignore
    new file:   README.md
    new file:   app.py
    new file:   collectors/__init__.py
    new file:   collectors/feodo.py
    new file:   collectors/http_downloader.py
    new file:   collectors/threatfox.py
    new file:   collectors/urlhaus.py
    new file:   config.py
    new file:   core/__init__.py
    new file:   core/correlator.py
    new file:   core/normalizer.py
    new file:   core/risk_scoring.py
    new file:   core/validator.py
    new file:   data/output/ti_results.json
    new file:   data/raw/feodo.csv
    new file:   data/raw/sample.csv
    new file:   data/raw/sample.json
    new file:   data/raw/sample.txt
    new file:   generators/__init__.py
    new file:   generators/blocklist.py
    new file:   generators/report.py
    new file:   parsers/__init__.py
    new file:   parsers/csv_parser.py
    new file:   parsers/feodo_parser.py
    new file:   parsers/json_parser.py
    new file:   parsers/txt_parser.py
    new file:   parsers/urlhaus_parser.py
    new file:   pipeline.py
    new file:   requirements.txt
    new file:   screenshots/01_pipeline_success.png
    new file:   static/style.css
    new file:   templates/index.html

(venv) PS C:\Users\nitin\OneDrive\Desktop\threat-intelligence-aggregator>
```

Figure 1. Successful end-to-end pipeline execution.

Appendix A - Project Evidence 2



The screenshot displays the Visual Studio Code interface with a project named 'threat-intelligence-aggregator'. The Explorer sidebar on the left shows the project structure, including folders like 'collectors', 'core', 'data', 'docs', 'generators', 'parsers', 'screenshots', 'static', 'templates', and 'venv', along with files like '.env', '.gitignore', 'app.py', 'config.py', 'pipeline.py', 'README.md', and 'requirements.txt'. The main editor window shows the 'pipeline.py' file with the following code:

```
62 def run_pipeline():
87     if item["risk_level"] == "MEDIUM"
88     )
89
90     low = sum(
91         1 for item in results
92         if item["risk_level"] == "LOW"
```

Below the code editor, the TERMINAL panel shows the output of running 'python pipeline.py' in a PowerShell environment. The output is as follows:

```
(venv) PS C:\Users\nitin\OneDrive\Desktop\threat-intelligence-aggregator> python pipeline.py

=====
THREAT INTELLIGENCE AGGREGATOR
=====

[+] Reading Feodo feed...
[+] Raw IOCs collected: 5
[+] Unique IOCs: 5

RISK SUMMARY
-----
HIGH   : 0
MEDIUM: 5
LOW    : 0

[+] Results saved to: C:\Users\nitin\OneDrive\Desktop\threat-intelligence-aggregator\data\output\ti_results.json

[+] Blocklists generated
[+] IPs       : 5
[+] Domains   : 0
[+] URLs      : 0
[+] Hashes    : 0
(venv) PS C:\Users\nitin\OneDrive\Desktop\threat-intelligence-aggregator>
```

Figure 2. Blocklist generation from processed indicators.

Appendix A - Project Evidence 3

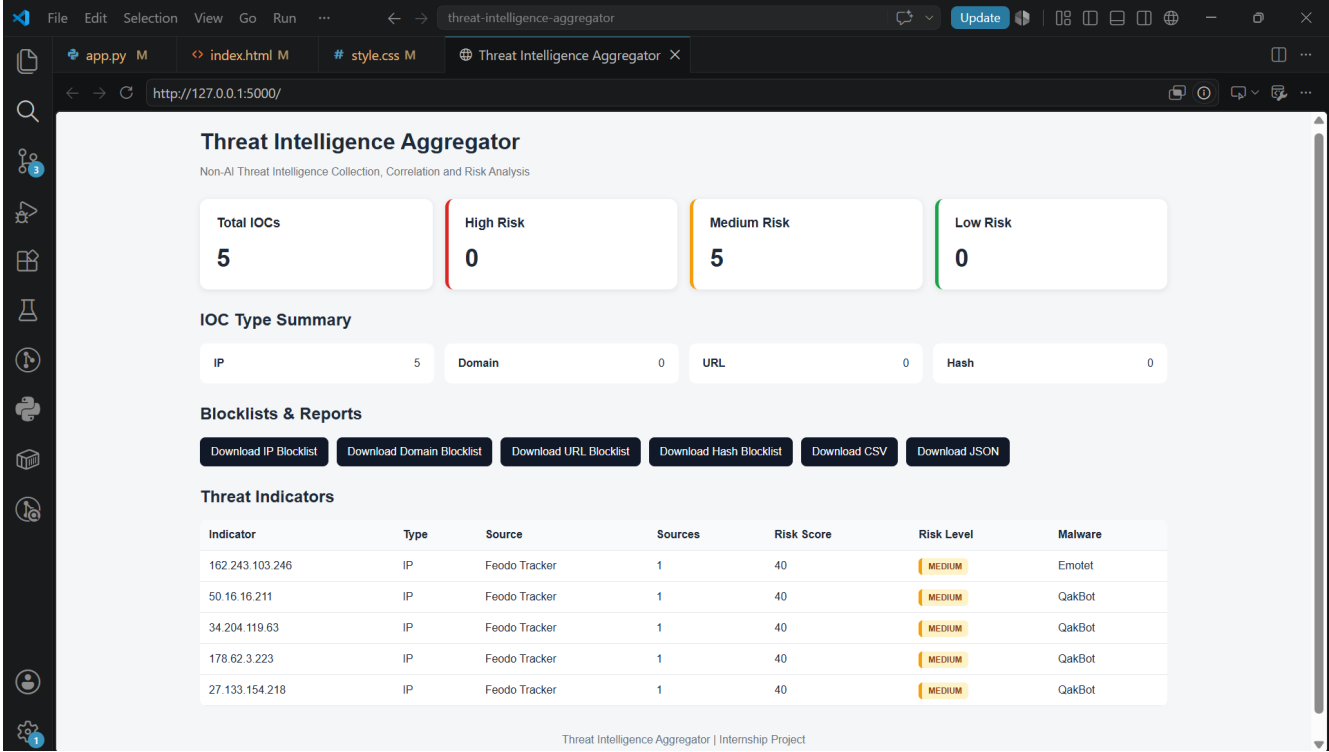
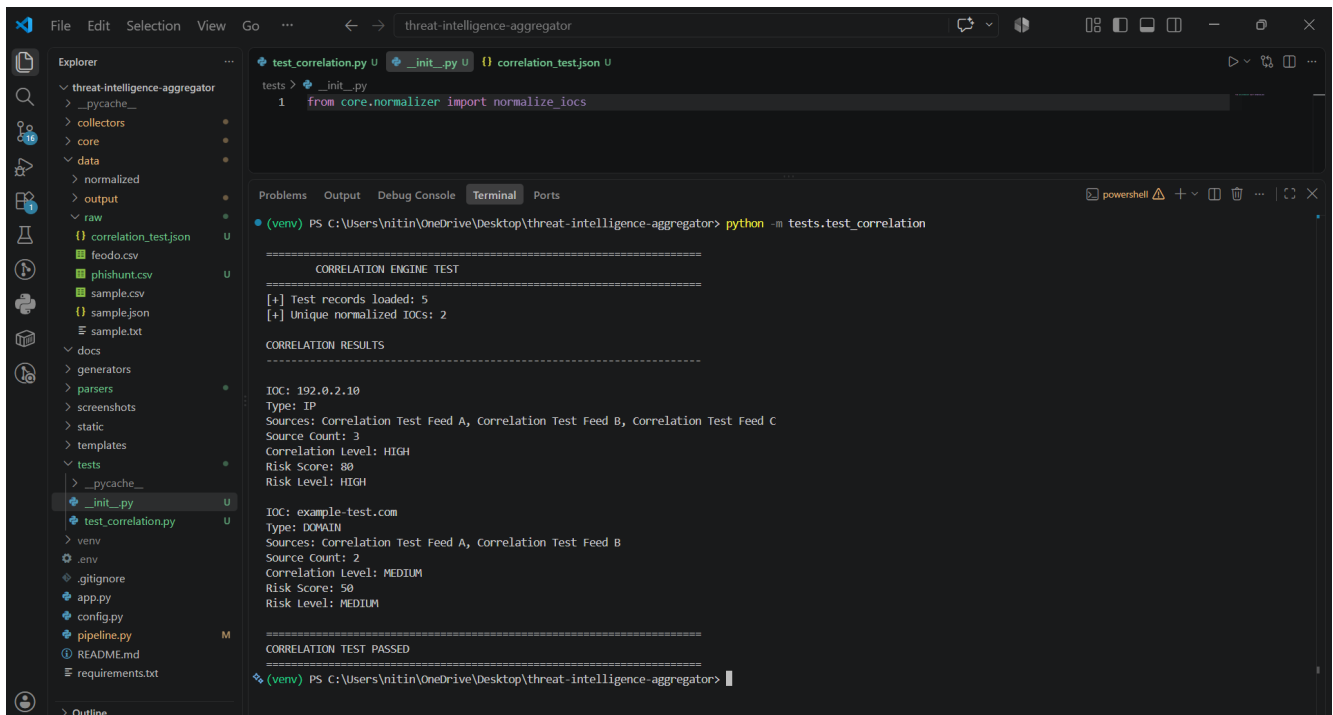


Figure 3. Flask web dashboard with IOC and risk statistics.

Appendix A - Project Evidence 4



The screenshot shows a Visual Studio Code editor window titled "threat-intelligence-aggregator". The Explorer sidebar on the left displays a file tree with folders like "collectors", "core", "data", "output", "raw", "correlation_testjson", "docs", "generators", "parsers", "screenshots", "static", "templates", "tests", and "venv". The "tests" folder is expanded, showing files like "test_correlation.py", "_init_.py", and "correlation_testjson". The main editor area shows the content of "test_correlation.py", which includes a line: `1 from core.normalizer import normalize_iocs`. Below the editor, the "Terminal" tab is active, displaying the output of a Python script execution. The output includes a "CORRELATION ENGINE TEST" section with test results and a "CORRELATION RESULTS" section showing two IOCs with their respective details.

```
tests > _init_.py
1 from core.normalizer import normalize_iocs

(venv) PS c:\Users\nitin\OneDrive\Desktop\threat-intelligence-aggregator> python -m tests.test_correlation

=====
CORRELATION ENGINE TEST
=====
[+] Test records loaded: 5
[+] Unique normalized IOCs: 2

CORRELATION RESULTS
=====

IOC: 192.0.2.10
Type: IP
Sources: Correlation Test Feed A, Correlation Test Feed B, Correlation Test Feed C
Source Count: 3
Correlation Level: HIGH
Risk Score: 80
Risk Level: HIGH

IOC: example-test.com
Type: DOMAIN
Sources: Correlation Test Feed A, Correlation Test Feed B
Source Count: 2
Correlation Level: MEDIUM
Risk Score: 50
Risk Level: MEDIUM

=====
CORRELATION TEST PASSED
=====
(venv) PS c:\Users\nitin\OneDrive\Desktop\threat-intelligence-aggregator>
```

Figure 4. Reproducible correlation-engine test showing multi-source correlation.

Appendix A - Project Evidence 5

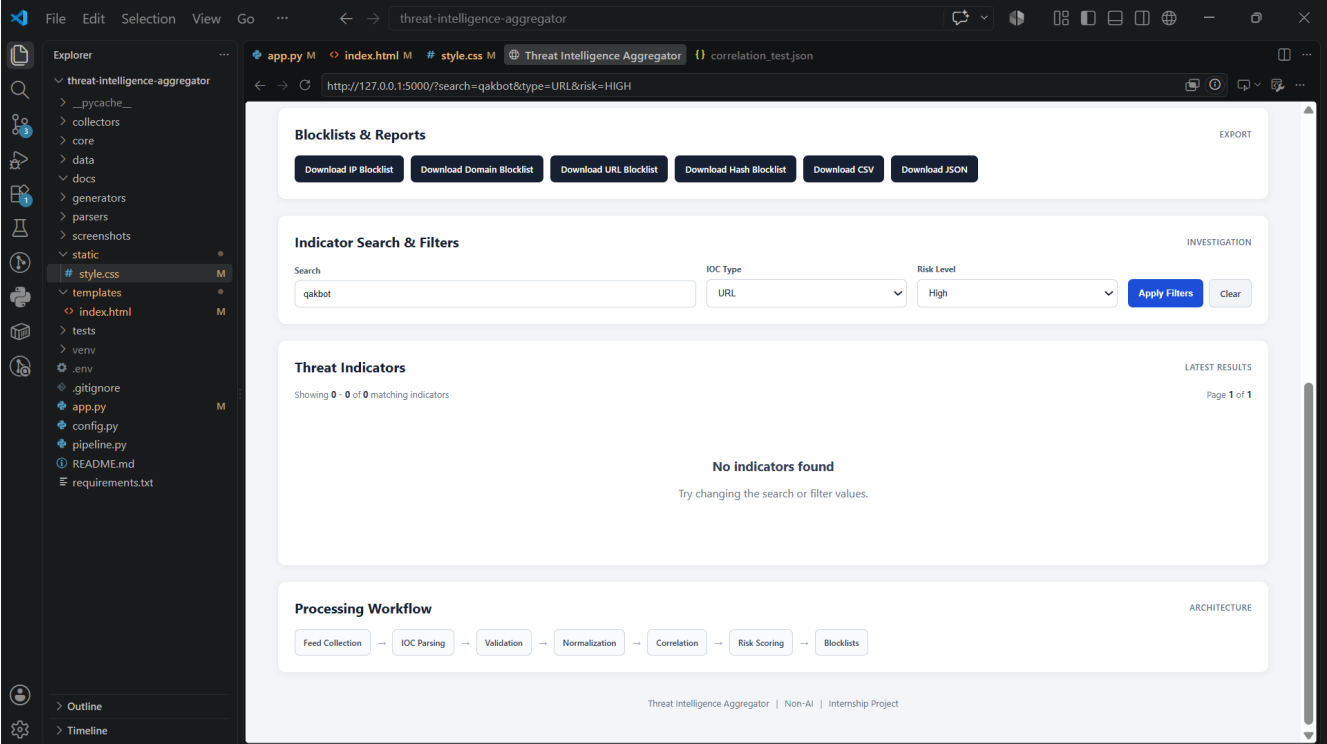


Figure 5. Dashboard search, IOC-type filtering, and risk filtering.

Appendix A - Project Evidence 6

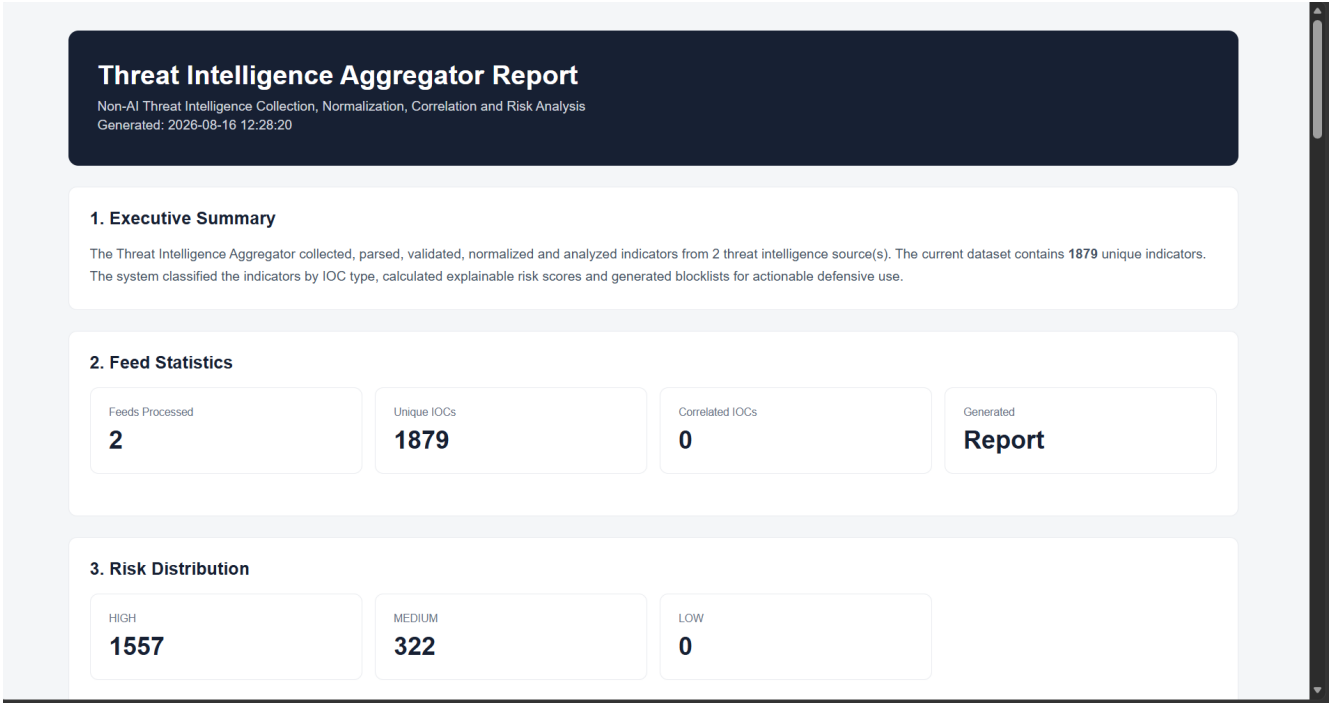


Figure 6. Generated threat-intelligence report.